Robotics & Internet-of-Things | جامعة الأمير سلطان PRINCE SULTAN UNIVERSITY

CS316: INTRODUCTION TO AI and DATA SCIENCE

CLASSIFICATION

Binary CLASSIFICATION

Classification – Heart Disease Prediction

- **Features Explained:**
  - **Age:** Patient's age in years.
  - **Sex:** Patient's gender (1 = male, 0 = female).
  - **Cp (Chest Pain Type):** Type of chest pain experienced (values from 1 to 4).
  - **Trestbps (Resting Blood Pressure):** Resting blood pressure in mm Hg.
  - **Chol (Serum Cholesterol):** Serum cholesterol in mg/dl.
  - **Fbs (Fasting Blood Sugar):** Fasting blood sugar > 120 mg/dl (1 = true, 0 = false).
  - **Restecg (Resting ECG Results):** Results of electrocardiogram at rest (0, 1, or 2).
  - **Thalach (Max Heart Rate Achieved):** Maximum heart rate achieved.
  - **Exang (Exercise Induced Angina):** Angina induced by exercise (1 = yes, 0 = no).
  - **Oldpeak:** ST depression induced by exercise relative to rest.
  - **Slope:** Slope of the peak exercise ST segment.
  - **Ca:** Number of major vessels colored by fluoroscopy.
  - **Thal:** Thalassemia (3 = normal; 6 = fixed defect; 7 = reversible defect).
- **Target Variable (Highlighted):**
  - **Num (Diagnosis of Heart Disease):** The degree of heart disease (0 = no presence, 1-4 indicate varying degrees of heart disease).

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num	Target
0	63	1	1	145	233	1	2	150	0	2.3	3	0.0	6.0	0	NO
1	67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	2	YES
2	67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1	YES
3	37	1	3	130	250	0	0	187	0	3.5	3	0.0	3.0	0	NO
4	41	0	2	130	204	0	2	172	0	1.4	1	0.0	3.0	0	NO

CLASSIFICATION

TARGET VARIABLE ENCODING

Encoding the Target Variable in Classification

- **Purpose of Encoding:**
  - Convert categorical variables into a numerical format suitable for algorithmic processing, enabling integration into mathematical models.
- **Label Encoding:**
  - **Definition:** Map each unique category to a unique integer.
  - **Example for 'Target':** Map 'NO' → 0 and 'YES' → 1.
  - **Mathematical Representation:** $\text{Target}_{\text{encoded}} = \begin{cases} 0 & \text{if Target} = \text{'NO'} \\ 1 & \text{if Target} = \text{'YES'} \end{cases}$

slope	ca	thal	num	Target
3	0.0	6.0	0	NO
2	3.0	3.0	2	YES
2	2.0	7.0	1	YES
3	0.0	3.0	0	NO
1	0.0	3.0	0	NO

One-Hot Encoding

- **One Hot Encoding:** Create 2 more cols, one for yes(1), one for no(0)
  - **Definition:** Convert each category value into a binary vector representing the presence (1) or absence (0) of the feature.
  - **Mathematical Representation:** Define indicator function $\mathbf{1}_{\text{category}}(x)$ for each category in the variable.
  - **Example for 'Target':**
    - 'NO' → [1, 0]
    - 'YES' → [0, 1]
  - **Vector Form:**
    - $\text{Target}_{\text{NO}} = \mathbf{1}_{\text{'NO'}}(\text{Target})$
    - $\text{Target}_{\text{YES}} = \mathbf{1}_{\text{'YES'}}(\text{Target})$

slope	ca	thal	num	Target
3	0.0	6.0	0	NO
2	3.0	3.0	2	YES
2	2.0	7.0	1	YES
3	0.0	3.0	0	NO
1	0.0	3.0	0	NO

Encoding the Target Variable in Python

- **Purpose of Label Encoding:**
  - Convert categorical text data into a model-readable numerical format.
- **Using Pandas for Label Encoding:**
  - **Code Example:**

```python
import pandas as pd
data = {'Target': ['NO', 'YES', 'NO', 'YES']}
df = pd.DataFrame(data)
df['Target_encoded'] = df['Target'].astype('category').cat.codes
print(df)
```

Copy code

	Target	Target_encoded
0	NO	0
1	YES	1
2	NO	0
3	YES	1

- **Key Points:**
  - Label encoding is straightforward but introduces a numerical order that may not exist, influencing some model types.

Encoding the Target Variable in Python

- **Purpose of One-Hot Encoding:**
  - Converts each category into a distinct binary column, which is crucial for models that do not assume any ordinal relationship between categories.
- **Updated Code for Scikit-learn One-Hot Encoding:**
  - Here's how to handle the one-hot encoding correctly using the latest Scikit-learn methods:

```python
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False)  # Corrected parameter usage
encoded = encoder.fit_transform(df[['Target']])  # Assuming df is predefined
df_encoded = pd.DataFrame(encoded, columns=encoder.get_feature_names_out(['Target
df = pd.concat([df, df_encoded], axis=1)
print(df)
```

- **Key Updates and Considerations:**
  - The use of `get_feature_names_out()` to retrieve column names from the encoder, replacing deprecated `get_feature_names()`.
  - Specifying `sparse_output=False` to handle the deprecation notice about the `sparse` parameter.

One-Hot Encoding with Scikit-learn

	Target	Target_NO	Target_YES
0	NO	1.0	0.0
1	YES	0.0	1.0
2	NO	1.0	0.0
3	YES	0.0	1.0

Encoding the Target Variable in Python

One-Hot Encoding with Pandas

- **Alternative Method with Pandas:**
  - Often simpler and more direct for handling within a DataFrame context.
- **Using Pandas for One-Hot Encoding:**
  - **Code Example:**

```python
df_one_hot = pd.get_dummies(df['Target'], prefix='Target')
df = pd.concat([df, df_one_hot], axis=1)
print(df)
```

- **Considerations:**
  - `get_dummies` is very efficient and directly integrates with pandas DataFrames, making it ideal for exploratory data analysis and preliminary data processing.

Each is incoded using binary code (either 0 or 1, not one having higher priority)
When using more numbers 1→5, the computer might misunderstand and say 5 is the highest number so higher priority which is wrong so we use the one-hot encoding do help lessen the misunderstanding

	Target	Target_NO	Target_YES	Target_NO	Target_YES
0	NO	1.0	0.0	True	False
1	YES	0.0	1.0	False	True
2	NO	1.0	0.0	True	False
3	YES	0.0	1.0	False	True

CLASSIFICATION

CLASSIFICATION TECHNIQUES:
Logistic regression technique

LOGISTIC REGRESSION

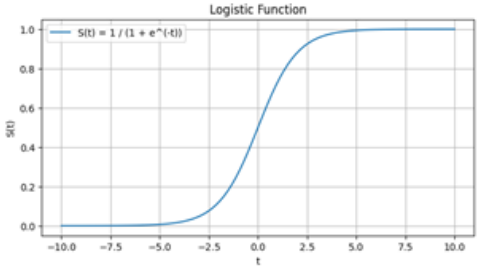
- **Definition:**
  - Logistic Regression is a statistical method for binary classification that models the probability of a binary response based on one or more predictor variables.
- **Mathematical Representation:**
  - It estimates probabilities using the logistic function, which is an S-shaped curve that maps any real-valued number into the (0, 1) interval, making it suitable for modeling probability.
- **Logistic Function:**
  - The logistic function is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

linear combination of the feature X like linearRegression()

  - Where z is the linear combination of the input features X and their corresponding coefficients β, expressed as:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$



- **Purpose:**
  - **Predictive Modelling:** Predicts the probability P that an observation falls into the category $Y = 1$ (positive class), as opposed to $Y = 0$ (negative class).
  - **Decision Boundary:** The decision boundary in logistic regression, which is set at $\sigma(z) = 0.5$, determines the threshold at which the probability output is converted into class labels.
- **Mathematical Goal:**
  - **Maximize Likelihood:** The parameters β are typically estimated using maximum likelihood estimation (MLE), which seeks to maximize the likelihood function, ensuring the best model coefficients that predict the observed outcomes.

Understanding the Logit in Logistic Regression

- **Logit to Sigmoid Relationship:**
  - The logistic regression model begins with a linear combination of input features x_i and their respective coefficients β_i:

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

  - The linear combination z then becomes the input to the logit function, which computes the log-odds of a positive outcome:

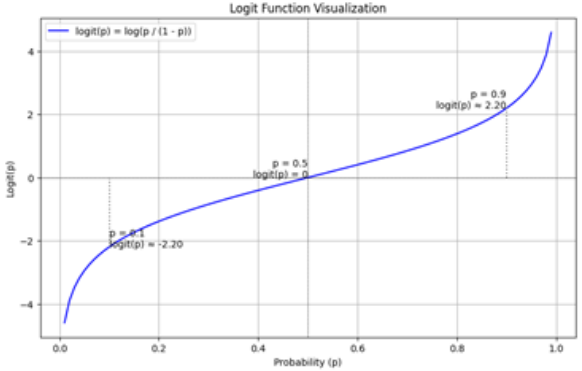
$$\text{logit}(P(Y = 1|X)) = \log\left(\frac{P(Y = 1|X)}{1 - P(Y = 1|X)}\right) = z$$

  - This relationship can be expressed more explicitly:

$$\text{logit}(P(Y = 1|X)) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$$

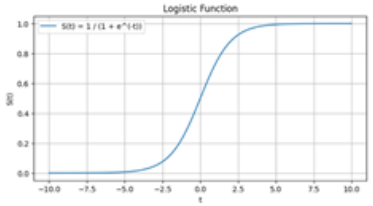
we need some sort of probability

  - Thus, the logit function provides a direct link between the linear combination of features and the log-odds of a positive outcome.



- **p = 0.1**: Here, the probability is low, and the logit value is negative, indicating a decreased likelihood of the positive outcome.
- **p = 0.5**: This point corresponds to equal odds (logit = 0), meaning the likelihood of positive and negative outcomes is balanced.
- **p = 0.9**: At this high probability, the logit value is positive and large, signifying a higher likelihood of the positive outcome.

Logit vs Sigmoid



- **Sigmoid (Logistic) Function:**
  - The sigmoid function is the inverse of the logit function and converts the linear combination z into a probability between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

  - Substituting the linear combination into the sigmoid function, we can predict the probability of the positive outcome as:

$$P(Y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n)}}$$

to extract the probability for z

  - This relationship translates the linear combination of features and coefficients into a probability of the positive outcome by applying the nonlinear sigmoid transformation.

CLASSIFICATION

LIKELIHOOD FUNCTION

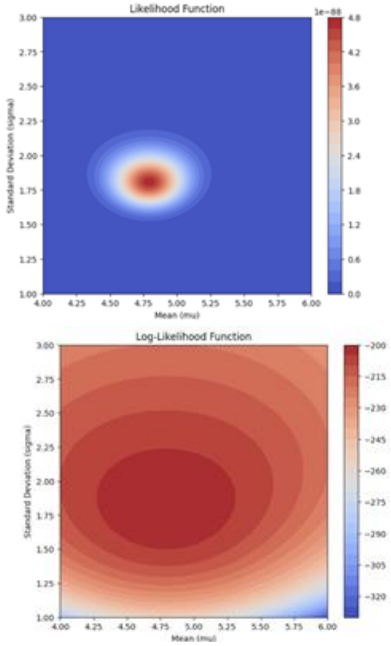
Understanding Likelihood and Log-Likelihood

- **Definition of Likelihood:**
  - The likelihood function measures the plausibility of a set of model parameters, given observed data.
  - Formally, given a statistical model $f(x|\theta)$ with parameters θ and data $X = \{x_1, x_2, \ldots, x_n\}$, the likelihood function is:

$$L(\theta|X) = \prod_{i=1}^{n} f(x_i|\theta)$$

- **Log-Likelihood:**
  - To simplify computation, the log-likelihood function is often used:

$$\ell(\theta) = \sum_{i=1}^{n} \log f(x_i|\theta)$$



- Both plots have regions with high values indicating parameter combinations that make the observed data most plausible.
- The log-likelihood plot often provides a more interpretable and computationally manageable way to understand the distribution of model parameters.

Understanding Likelihood and Log Likelihood in Logistic Regression

1. **Likelihood**
   - **Definition**: Measures how probable the observed outcomes are, given the model's predictions.
   - **Purpose**: Higher likelihood indicates better model fit to the actual data.
   - **Usage**: Used to assess and maximize the accuracy of the model predictions.
2. **Log Likelihood**
   - **Definition**: The natural logarithm of the likelihood.
   - **Purpose**: Simplifies calculations by transforming products of probabilities into sums.
   - **Benefits**:
     - Easier to maximize during model optimization.
     - Improves numerical stability and handling in computations.

CLASSIFICATION

TRAINING A LOGISTIC REGRESSION CLASSIFICATION MODEL

Preparing the Heart Disease Dataset

Input Features

Output

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal	num	Target
0	63	1	1	145	233	1	2	150	0	2.3	3	0.0	6.0	0	NO
1	67	1	4	160	286	0	2	108	1	1.5	2	3.0	3.0	2	YES
2	67	1	4	120	229	0	2	129	1	2.6	2	2.0	7.0	1	YES
3	37	1	3	130	250	0	0	187	0	3.5	3	0.0	3.0	0	NO
4	41	0	2	130	204	0	2	172	0	1.4	1	0.0	3.0	0	NO

Convert/encode the input to categorical features

Separate Features from Target

- **Data Loading:**
  - Import dataset from a URL into a pandas DataFrame.
- **Target Variable Creation:**
  - Generate a new binary 'Target' column:
    - 'YES' for heart disease ($\text{num} > 0$)
    - 'NO' for no heart disease ($\text{num} = 0$)
- **Data Segmentation:**
  - Separate the dataset into features (X) and target (y).
- **Tools and Functions:**
  - `pd.read_csv()`: Reads data from URL.
  - `.apply()`: Applies a lambda function for binary classification.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import log_loss
from sklearn.preprocessing import OneHotEncoder

# URL of the dataset
url = "https://www.riotu-lab.org/cs313/heart_data.csv"

# Read the dataset from the URL
df = pd.read_csv(url)

# Add a new column 'Response' based on the 'num' column
df['Target'] = df['num'].apply(lambda x: 'YES' if x > 0 else 'NO')

# Display the first few rows of the dataset
print("Sample dataset:")
#print(df.head())

# Separate features and target variable
X = df.drop(['num', 'Target'], axis=1)
y = df['Target']
```

One-Hot Encoding Categorical Features

- **Purpose:**
  - Convert categorical variables into a form that can be provided to machine learning algorithms to improve model performance.
- **Process:**
  - **Identify Categorical Features:**
    - List of categorical variables: 'sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal'.
  - **Apply One-Hot Encoding:**
    - Use `OneHotEncoder` to transform these features into binary columns.
    - Handle unknown categories by ignoring them (`handle_unknown='ignore'`).
- **Integration:**
  - Create a new DataFrame `X_encoded` from the encoded data.
  - Name new columns using the original feature names to maintain clarity.
  - Merge encoded features back with the non-categorical features.
- **Key Functions Used:**
  - `OneHotEncoder()`: Transforms categorical values into a binary matrix.
  - `fit_transform()`: Fits the encoder to the data and transforms it.
  - `get_feature_names_out()`: Retrieves column names for the new binary matrix.
  - `pd.concat()`: Combines the original numeric features with the new encoded data.

```
# Perform one-hot encoding on categorical features
categorical_features = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']
encoder = OneHotEncoder(handle_unknown='ignore')
X_encoded = pd.DataFrame(encoder.fit_transform(X[categorical_features]).toarray())
X_encoded.columns = encoder.get_feature_names_out(categorical_features)
X_encoded = pd.concat([X_encoded, X.drop(categorical_features, axis=1)], axis=1)
```

```
[50]  X_encoded.columns = encoder.get_feature_names_out(categorical_features)
      X_encoded.columns
```

```
Index(['sex_0', 'sex_1', 'cp_1', 'cp_2', 'cp_3', 'cp_4', 'fbs_0', 'fbs_1',
       'restecg_0', 'restecg_1', 'restecg_2', 'exang_0', 'exang_1', 'slope_1',
       'slope_2', 'slope_3', 'ca_0.0', 'ca_1.0', 'ca_2.0', 'ca_3.0', 'ca_nan',
       'thal_3.0', 'thal_6.0', 'thal_7.0', 'thal_nan'],
      dtype='object')
```

```
X_encoded
```

	0	1	2	3	4	5	6	7	8	9	...	15	16	17	18	19	20	21	22	23	24
0	0.0	1.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	...	1.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
1	0.0	1.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	0.0	0.0
2	0.0	1.0	0.0	0.0	0.0	1.0	1.0	0.0	0.0	0.0	...	0.0	0.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0
3	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0	...	1.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0
4	1.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
298	0.0	1.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	...	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
299	0.0	1.0	0.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	...	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	1.0	0.0
300	0.0	1.0	0.0	0.0	0.0	1.0	1.0	0.0	1.0	0.0	...	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	1.0	0.0
301	1.0	0.0	0.0	1.0	0.0	0.0	1.0	0.0	0.0	0.0	...	0.0	0.0	1.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0
302	0.0	1.0	0.0	0.0	1.0	0.0	1.0	0.0	1.0	0.0	...	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0

Training and Testing the Logistic Regression Model

- **Data Splitting:**
  - **Objective:** Partition the data into training and testing subsets to evaluate the model's performance on unseen data.
  - **Method:**
    - Use `train_test_split` from scikit-learn.
    - Assign 20% of the data to the test set (`test_size=0.2`).
    - Ensure consistency across runs with `random_state=42`.
- **Model Training:**
  - **Logistic Regression Setup:**
    - Initialize the Logistic Regression model.
  - **Training Process:**
    - Fit the model on the training data using `model.fit(X_train, y_train)`.
- **Key Functions Used:**
  - `train_test_split()`: Splits the dataset into separate training and testing sets.
  - `LogisticRegression()`: Creates a logistic regression model.
  - `fit()`: Trains the logistic regression model on the training data.

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X_encoded, y,
                                                    test_size=0.2,
                                                    random_state=42)

# Train a logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)
```

Evaluating Model Performance

- **Accuracy as a Performance Metric:**
  - **Purpose:** Evaluate the accuracy of the logistic regression model on the test dataset.
  - **Method:** `model.score(X_test, y_test)` calculates the accuracy, the proportion of correct predictions.
- **Code and Output:**
  - **Python Execution:**

```python
log_likelihood = model.score(X_test, y_test)  # Note: This is act
print("Final log-likelihood:", log_likelihood)
```

  - **Interpretation:** The reported score, labeled as 'log-likelihood', is actually the accuracy of the model, which is approximately 0.869. This means the model correctly predicts the outcome for 86.9% of the test cases.
- **Key Python Function:**
  - `model.score()`: Computes the accuracy, not the log-likelihood, contrary to what the variable name suggests.

```
1  # Get the final log-likelihood
2  log_likelihood = model.score(X_test, y_test)
3  print("Final log-likelihood:", log_likelihood)
```

returns some sort of probability to see x belongs to which class

```
Final log-likelihood: 0.8688524590163934
```

CLASSIFICATION

PREDICTION WITH A LOGISTIC REGRESSION MODEL

Making Predictions with Logistic Regression

```python
# Make predictions on the test set
y_pred = model.predict(X_test)

# Display a sample of predictions
print("Sample predictions:")
pd.DataFrame({'Actual': y_test, 'Predicted': y_pred}).head(10)
```

```
Sample predictions:
```

	Actual	Predicted
179	NO	NO
228	YES	YES
111	YES	YES
246	YES	YES
60	YES	YES
9	YES	YES
119	YES	YES
223	YES	YES
268	YES	NO
33	NO	YES

- **Model Prediction:**
  - **Objective:** Use the trained logistic regression model to make predictions on the test dataset.
  - **Process:**
    - Predict using `` `model.predict(X_test)` ``, which classifies each instance in the test set based on the learned parameters.
- **Displaying Predictions:**
  - **Sample Output:**
    - Combine actual and predicted labels in a DataFrame to compare results visually.
    - **Code Snippet:**

```python
y_pred = model.predict(X_test)
sample_predictions = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred}).head
print("Sample predictions:", sample_predictions)
```

- **Key Functions Used:**
  - `` `model.predict()` ``: Generates label predictions for the given input data.
  - `` `pd.DataFrame()` ``: Constructs a DataFrame from the actual and predicted labels for easy comparison.

Making Predictions with Logistic Regression

- **Beyond Accuracy:**
  - What limitations does relying solely on accuracy present?
  - How might different thresholds affect model evaluation?
- **Understanding Errors:**
  - What can precision, recall, and F1-score tell us that accuracy cannot?
  - How does the confusion matrix provide deeper insights into model performance?
- **Handling Imbalance:**
  - How should we adjust our evaluation for imbalanced datasets?
  - What methods like SMOTE or cost-sensitive learning could be applied?
- **Testing Generalization:**
  - How robust is the model against new, unseen data?
  - What validation techniques ensure the model generalizes well?
- **Deep Dive into Model Assessment:**
  - Why is cross-validation critical for assessing model reliability?
  - How does the ROC-AUC curve help in understanding class differentiation?

CLASSIFICATION

MODEL EVALUATION IN PYTHON

Model Evaluation in Python – Individual Metrics

- **Using scikit-learn for Model Evaluation**
  - **Libraries and Functions:**
    - Utilize `sklearn.metrics` for comprehensive model evaluation tools.
    - Functions used: `accuracy_score`, `precision_score`, `recall_score`, `f1_score`.
- **Calculation of Metrics**
  - **Accuracy:**
    - `accuracy = accuracy_score(y_test, y_pred)`
    - Represents overall correctness of the model.
  - **Precision:**
    - `precision = precision_score(y_test, y_pred, pos_label='YES')`
    - Indicates the accuracy of positive predictions.
  - **Recall:**
    - `recall = recall_score(y_test, y_pred, pos_label='YES')`
    - Measures the model's ability to identify all relevant instances.
  - **F1 Score:**
    - `f1 = f1_score(y_test, y_pred, pos_label='YES')`
    - Harmonic mean of precision and recall, balancing both metrics.

we need to specify the positive class

```python
from sklearn.metrics import confusion_matrix, accuracy_score
from sklearn.metrics import precision_score, recall_score
from sklearn.metrics import f1_score, classification_report
# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, pos_label='YES')
recall = recall_score(y_test, y_pred, pos_label='YES')
f1 = f1_score(y_test, y_pred, pos_label='YES')

# Print basic metrics
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")
```

```
Accuracy: 0.84
Precision: 0.84
Recall: 0.84
F1 Score: 0.84
```

Classification Report & Confusion Matrix

- **Python Code for Model Evaluation**
  - Use `classification_report` and `confusion_matrix` from `sklearn.metrics` for detailed performance analysis.
- **Classification Report**
  - **Math Formulas:**
    - Precision: $\text{Precision} = \frac{TP}{TP+FP}$
    - Recall: $\text{Recall} = \frac{TP}{TP+FN}$
    - F1-Score: $\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$
  - **Code**: `report = classification_report(y_test, y_pred, target_names=model.classes_)`
  - Outputs precision, recall, and F1-score for each class.
- **Confusion Matrix**
  - **Visualization with Python:**
    - **Code:**

```python
cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
cm_df = pd.DataFrame(cm, index=model.classes_, columns=model.classes_)
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues', annot_kws={"size": 16})
```

    - Displays predicted vs actual classifications in a heatmap format.

```python
# Generate and print the classification report
report = classification_report(y_test, y_pred, target_names=model.classes_)
print("Classification Report:")
print(report)

# Compute confusion matrix to evaluate the accuracy of a classification
cm = confusion_matrix(y_test, y_pred, labels=model.classes_)
cm_df = pd.DataFrame(cm, index=model.classes_, columns=model.classes_)

# Plotting the confusion matrix
plt.figure(figsize=(10, 8))
sns.heatmap(cm_df, annot=True, fmt='d', cmap='Blues', annot_kws={"size": 16})
plt.title('Confusion Matrix', size=20)
plt.xlabel('Predicted', size=18)
plt.ylabel('Actual', size=18)
plt.xticks(fontsize=14)
plt.yticks(fontsize=14)
plt.show()
```

```
Classification Report:
              precision    recall  f1-score   support

          NO       0.83      0.83      0.83        29
         YES       0.84      0.84      0.84        32

    accuracy                           0.84        61
   macro avg       0.84      0.84      0.84        61
weighted avg       0.84      0.84      0.84        61
```

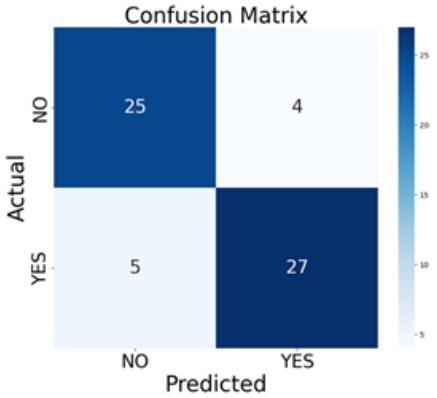
CLASSIFICATION

RESULTS INTERPRETATION AND INSIGHTS

Interpretation of Heart Disease Classification Results

- **Classification Report Overview**
  - **Precision**:
    - 'NO': 0.83 - 83% of the predictions for non-disease cases were correct.
    - 'YES': 0.84 - 84% of the predictions for disease cases were correct.
  - **Recall**:
    - 'NO': 0.83 - Of all actual non-disease cases, 83% were correctly identified.
    - 'YES': 0.84 - Of all actual disease cases, 84% were correctly identified.
  - **F1-Score**:
    - 'NO': 0.83
    - 'YES': 0.84
    - Indicates balanced precision and recall for both classes.
- **Accuracy of the Model**
  - **0.84 (84%)**: Reflects that the overall predictions (both 'YES' and 'NO') are correct 84% of the time.

In a classification report, "support" refers to the number of actual occurrences of each class in the dataset used for testing the model.




```
Classification Report:
              precision    recall  f1-score   support

          NO       0.83      0.83      0.83        29
         YES       0.84      0.84      0.84        32

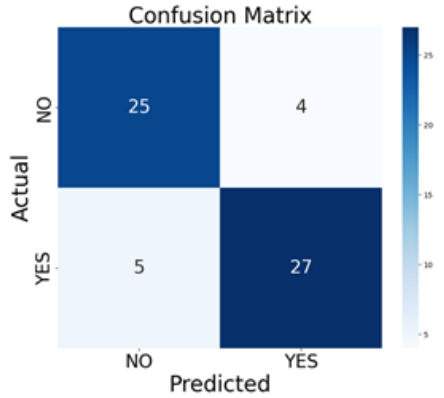
    accuracy                           0.84        61
   macro avg       0.84      0.84      0.84        61
weighted avg       0.84      0.84      0.84        61
```

good classifier

however if the impact for the wrong classification is high then it needs to be better for critical cases, the error would be more than 16%

Technical Insights and Optimization Strategies

- **Model Evaluation Metrics Review**
  - **Performance Consistency**: Both macro and weighted averages report at 0.84, reflecting homogeneous accuracy across different class weights and sample distributions.
  - **Metric Uniformity**: Precision, recall, and F1-scores are equally strong (0.84), highlighting a balanced approach to type I and II error management.
- **Strategic Enhancement Priorities**
  - **Optimizing Sensitivity**: Despite high scores, enhancing recall is paramount to ensure no critical condition goes undetected.
  - **Algorithmic Refinements**: Investigate ensemble methods, advanced resampling techniques, or cost-sensitive learning to reduce false negatives.
- **Implications for Medical Diagnostics**
  - **Decision Support Reliability**: The model's robustness is crucial in supporting high-stakes clinical decisions.
  - **Future Adjustments**: Prioritize adjustments that improve recall, using precision-recall trade-off analyses to ensure optimal patient outcomes without inducing unnecessary medical procedures.



Classification Report:

	precision	recall	f1-score	support
NO	0.83	0.83	0.83	29
YES	0.84	0.84	0.84	32
accuracy			0.84	61
macro avg	0.84	0.84	0.84	61
weighted avg	0.84	0.84	0.84	61

CLASSIFICATION

ROC and AUC

Limitation of the Classification Report

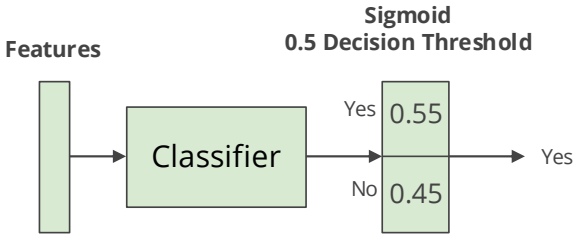
- **Limitations of Classification Metrics**
  - **Precision and Recall Limitations:**
    - Dependent on a specific **decision threshold.**
    - Do not provide insight into classification performance across varying thresholds.
  - **Inadequate for Imbalanced Datasets:**
    - Metrics can be biased towards the **majority class**, misleading model evaluation.

high threshold means high tolerance

Ex: threshold is 0.9 if some is sick

if someone has 0.8 they will be shown

as healthy even though the percentage is high



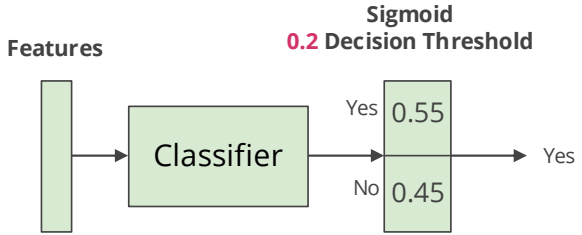
Classification Report:

	precision	recall	f1-score	support
NO	0.83	0.83	0.83	29
YES	0.84	0.84	0.84	32
accuracy			0.84	61
macro avg	0.84	0.84	0.84	61
weighted avg	0.84	0.84	0.84	61

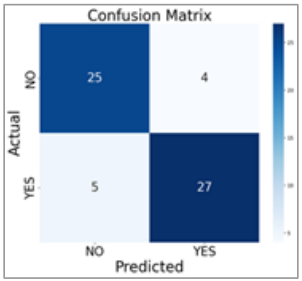
Limitation of the Classification Report

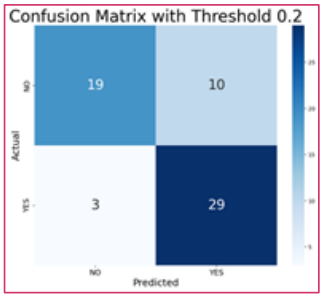


- **Limitations of Classification Metrics**
  - **Precision and Recall Limitations:**
    - Dependent on a specific **decision threshold.**
      - *Precision drops from **0.85 to 0.74** when the decision threshold is lowered from **0.5 to 0.2**, illustrating sensitivity to threshold settings.*
    - Do not provide insight into classification performance across varying thresholds.
  - **Inadequate for Imbalanced Datasets:**
    - Metrics can be biased towards the **majority class**, misleading model evaluation.




```
New Threshold - Accuracy: 0.79
New Threshold - Precision: 0.74
New Threshold - Recall: 0.91
```

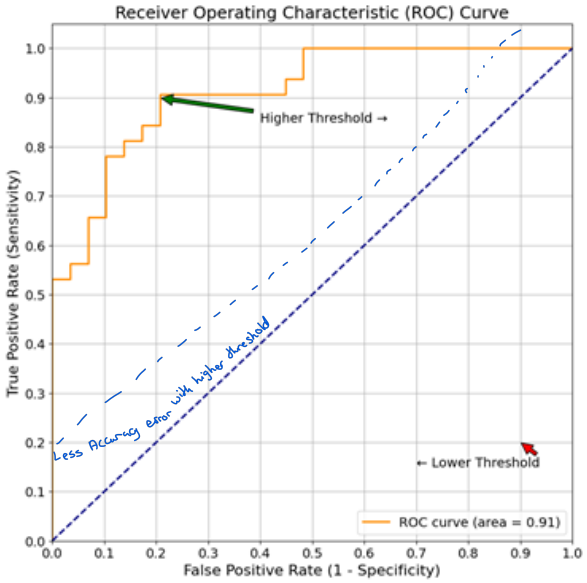




ROC Analysis in Model Evaluation

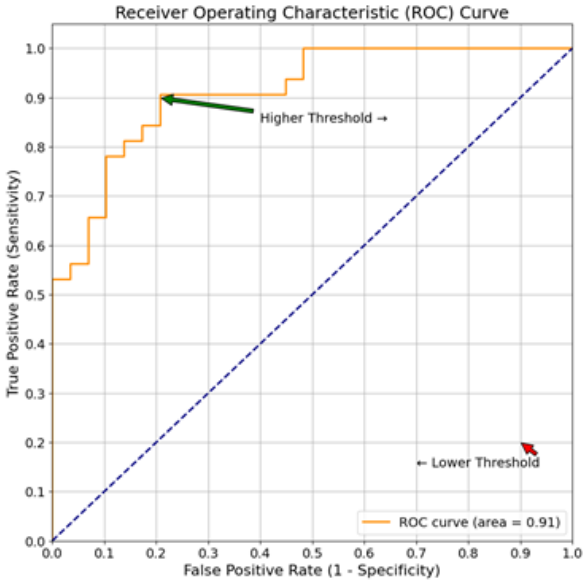
ROC Curve defines the performance of the model

- **Understanding the ROC Curve**
  - **Curve Overview**: The orange ROC curve shows how sensitivity (TPR) and 1-specificity (FPR) vary at different threshold levels.
  - **Threshold Movement**: As the threshold for classifying a patient as having the disease is lowered (moving from right to left), both the sensitivity and false positive rate increase.
- **ROC Curve Insights**
  - **Steep Initial Ascent**: Indicates rapid gain in sensitivity with minimal increase in false positives, important for early disease detection.
  - **Curve Plateau**: Shows diminishing returns in sensitivity, highlighting limits in model performance as threshold decreases.
- **AUC Value: 0.91**
  Area under the curve (we want a larger area => better model)
  - Indicates excellent discrimination ability, essential for reliable disease diagnosis in healthcare.
  - Suggests that the model correctly ranks a positive instance over a negative one in 91% of comparisons.
- **Healthcare Application**
  - **Optimal Threshold Setting**: Helps determine the most effective balance between detecting disease cases (sensitivity) and minimizing false alarms (specificity).
  - **Model Selection**: Prefer models with higher AUC for critical diagnostic applications to ensure accurate patient outcomes.



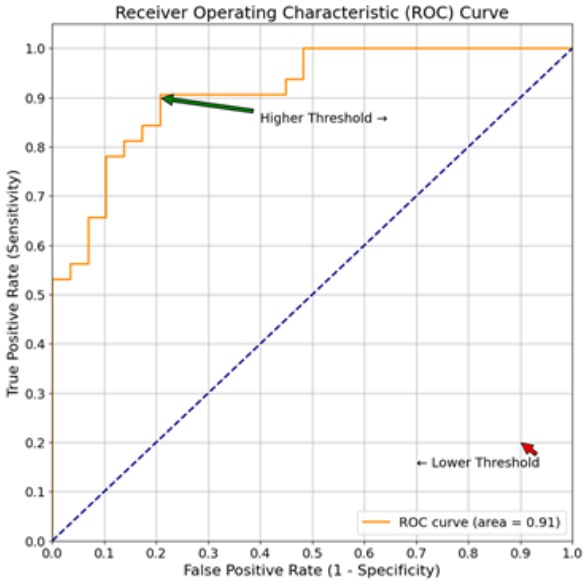
ROC Analysis in Model Evaluation

- **Motivation: Overcoming Limitations**
  - Traditional metrics are threshold-specific and fail to depict model performance comprehensively, especially under variable conditions and class imbalances.
- **ROC Curve Defined**
  - **ROC Curve**: Plots True Positive Rate (TPR) against False Positive Rate (FPR) to show model performance across thresholds.
  - **Purpose**: Evaluates the ability to distinguish between classes under varying conditions.
- **Mathematical Formulation**
  - **True Positive Rate (TPR)**: $\text{Sensitivity} = \frac{TP}{TP+FN}$
  - **False Positive Rate (FPR)**: $1 - \text{Specificity} = \frac{FP}{FP+TN}$
- **Utility as a Classification Metric**
  - Enables comprehensive evaluation by illustrating the trade-off between sensitivity and false alarms, crucial for determining the most effective operational threshold.



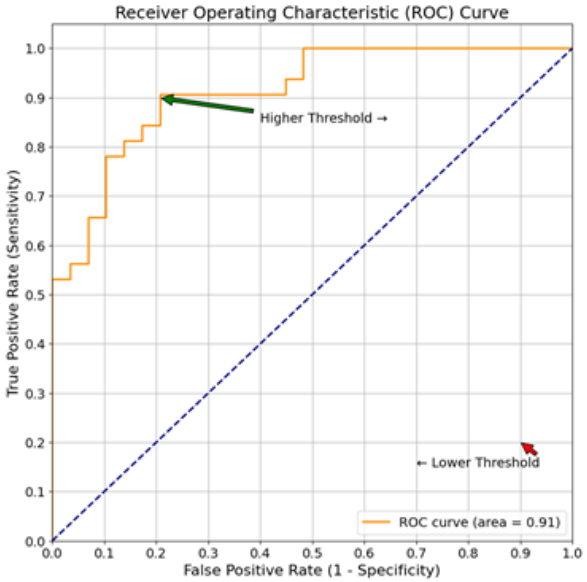
Plotting the ROC Curve: An Intuitive Guide

- **Objective**: Illustrate a model's discrimination ability across various thresholds.
- **Algorithm Steps**:
  1. **Initialize**: Set the highest threshold (typically 1) where no positives are predicted; start at $(0, 0)$ on the ROC plot.
  2. **Calculate for Each Threshold**:
     - Decrement threshold from 1 to 0.
     - **True Positive Rate (TPR)**: $\mathrm{TPR} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}}$
     - **False Positive Rate (FPR)**: $\mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP}+\mathrm{TN}}$
     - Compute $\mathrm{TP}, \mathrm{FP}, \mathrm{TN}, \mathrm{FN}$ based on each threshold.
  3. **Plot**: For each threshold, plot (FPR, TPR) to trace the ROC curve from the bottom left (most conservative) to the top right (least conservative).
- **Evaluate**:
  - Closer the curve approaches the top left corner, the better the classifier.
  - The area under the curve (AUC) measures overall effectiveness.



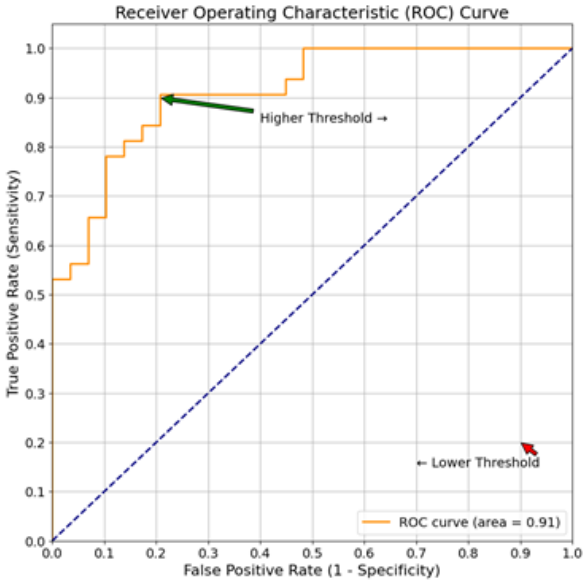
Interpreting the ROC Curve Intuitively

- **Key Aspects of ROC Curve Interpretation**
  - **Above the Diagonal**: Indicates better-than-random performance. The higher and closer to the top left corner, the better the model's sensitivity and specificity.
  - **Shape and Direction**: A steep initial ascent suggests high sensitivity for a low false positive rate—ideal in many practical applications.
  - **Area Under Curve (AUC)**: A direct measure of model accuracy; higher AUC values (closer to 1.0) signify better overall model performance.



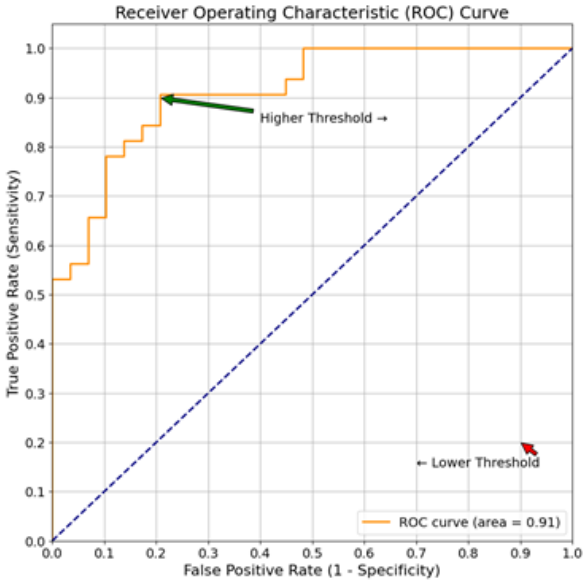
Understanding the Diagonal in ROC Curve Plots

- **Baseline Performance**
  - The diagonal line represents a random classifier, serving as a baseline for comparison.
- **Indication of No Discrimination**
  - Slope of 1 (45-degree angle) indicates no ability to distinguish between classes; akin to flipping a coin.
- **Reference for Model Evaluation**
  - Effective classifiers exhibit ROC curves above this line, showing superior discrimination.
  - Curves below suggest performance worse than random, signaling model issues.



Area Under the Curve

- **What is AUC?**
  - AUC, or Area Under the ROC Curve, quantifies a model's ability to correctly classify positives and negatives at various threshold levels.
- **Why is AUC Important?**
  - **Comprehensive Measure**: It evaluates model performance over all possible classification thresholds, not just one.
  - **Model Comparison**: Simplifies comparing the effectiveness of different models by providing a single value ranging from 0.5 (no discrimination) to 1.0 (perfect discrimination).
- **Practical Use**
  - **Optimal Selection**: A higher AUC indicates a model with better overall predictive accuracy, essential in applications like medical diagnosis or fraud detection.



Area Under the Curve

- **Definition of AUC**
  - Measures the area underneath the ROC curve; evaluates a model's ability to distinguish between classes across all thresholds.
- **Significance of AUC**
  - **Scalar Value Interpretation**:
    - **1.0**: Perfect discrimination.
    - **0.5**: No better than random.
    - **<0.5**: Worse than random, indicates potential model issues.
  - **Advantages**:
    - Threshold-independent, providing a comprehensive overview.
    - Enables straightforward comparisons between different models.

